

CarRacing PPO 实战 · Part 4

崩溃、么蛾子与 34 步的 debug 长征

手写 PPO 训练日志 · 从 eval 452 的假象到 iter60 的走线完美 · 附全部失败复盘

本篇导读

- 0. 背景:我们站在哪里
- 1. 第一次假胜利:eval 452 是打转攒出来的分
- 2. reward misspec 回旋镖:出界惩罚 -1 把车训成了"躺平"
- 3. KL 爆炸与 critic 脱节:冲高后崩溃的完整机制
- 4. log_std 的起点:std=1 是一切摇晃的根源
- 5. S 弯攻坚:从蹭草地到出弯回正的临门一脚
- 6. checkpoint 的四个坑:resume / opt / eval 跨步数 / KL 保护失效
- 6.5 更多致命坑:warmup卡死 / 续训推歪 / eval_every改轨迹 / 扰动实验 / truncated偏差
- 7. 2000 步的么蛾子:步数宽松如何喂养 reward hacking
- 7.5 训练中的"吸引子动物园":打转/躺平/贴草边/横跳/画圆/反转...
- 8. 两个迟到的洞察:智能采样 与 训练cost/评测reward 分离
- 9. 方法论沉淀:数字 + gif 双读、单变量、每步存档
- 10. 最终成果:从崩溃到 iter60"走线完美"的完整路径(附 gif / 模型链接)

0. 背景:我们站在哪里

Part 1-3 里,我们从零手写了 PPO 的三个核心——连续高斯策略、GAE、clip surrogate loss——并让它在 CarRacing-v3 上跑了起来。Part 4 记录的是最难、也最有价值的一段:把一个"能跑但会崩"的 agent,一步步 debug 成"走线完美"的过程。

这一段几乎全是失败。但正是这些失败,让我第一次真正理解了 RL 为什么脆弱、么蛾子从哪来、以及"数字好看"和"策略真的好"之间的鸿沟。所以这篇会把每一个失败的现象、诊断、根因、教训都完整写下来——因为这些坑,做 CarRacing 的人几乎都会踩。

核心配置(贯穿全篇)

网络:Nature-DQN CNN(Conv $8 \times 4 \rightarrow 4 \times 2 \rightarrow 3 \times 1 \rightarrow \text{FC512}$)+ 连续高斯策略头 + value 头。

算法:PPO,epochs=1,mb=1024,clip=0.2,GAE($\gamma=0.99$, $\lambda=0.95$),KL 早停。

环境:CarRacing-v3,4 帧堆叠,原始 reward(过 tile +3.4 / 帧 -0.1 / 飞出场地 -100)。

评测:5 条固定 seed 赛道 $\times$ 确定性动作(高斯均值) $\times$ 平均。

1. 第一次假胜利:eval 452 是打转攒出来的分

现象:早期某个 run, eval 一路从 -101 爬到 447, 看起来马上就要 solved (Car Racing 满分约 900, 官方 solved 线 900)。当时非常兴奋。

崩溃:it100 冲到 447 后, it110-120 之间断崖式跌到 -79, 之后永远卡在 -82。

更诡异的是:崩溃当轮的 kl / clipfrac / explained variance 全部正常——在所有已有指标上, 那次"致命更新"都是无害的。

诊断:拆帧看 gif, 而不是盯数字

数字查不出病因, 于是我把 it110 和 it120 的评测 gif 逐帧拆开。真相触目惊心:车在草地上原地打转, 身后画着一圈圈螺旋轨迹。

再回去核对环境的 reward 机制, 发现关键事实:

- reward 来自访问新 tile (每个 $+3.4 = 1000/N$), 不是"在赛道上"。
- reward $\propto$ 速度 (跑得快 $\rightarrow$ 单位时间吃更多 tile)。
- 高速 $\rightarrow$ 无法过弯 $\rightarrow$ 飞出赛道。

根因:eval 447 不是"会开车", 是"在冲出赛道之前那段高速直道上, 把 tile 分攒够了"。冲出去打转后不再得分, 但之前攒的分还在。452/447 这个数字是"莽夫式猛冲"的产物——学会了加速这一招, 一遇到极限(高速过弯)就崩, 而且崩了回不来(critic 也崩了)。

教训 1:eval 高 $\neq$ 策略好。带高方差(± 143)的单次峰值, 极可能是"运气 + 取巧"攒出来的。此后我坚持一条:数字看趋势和方差, gif 看真实行为, 两者对不上时, 信 gif。

2. reward misspec 回旋镖:出界惩罚 -1 把车训成了"躺平"

既然是"冲出赛道进草地"导致的崩溃, 直觉反应是:加个出界惩罚, 让它别冲出去。于是我给"车轮全在草地上"加了 -1/帧 的惩罚。

灾难性后果:agent 学会了原地不动(躺平)。eval -1045。

机制:出界 -1 太重 $\rightarrow$ "不动/待在原地"成了最安全的策略(不出界就不扣那 -1) $\rightarrow$ std 从 1.0 纹丝不动, 策略锁死在"躺平"这个吸收态。

把惩罚从 -1 降到 -0.3, 躺平缓解了, 但又出现新问题:

ev 涨但 eval 卡死:critic 的 explained variance 一路爬(说明 value 在学), 但 eval 死死卡在 -82, std 永远是 1.0(策略根本没在学)。

机制——"大数吃小数":出界惩罚让 return 方差爆炸(在赛道的 episode ≈ -50 , 进草地的 ≈ -800)。value loss 的量级(100)远远压过 policy loss(1) $\rightarrow$ policy 的梯度信号被淹没 $\rightarrow$ std 不降, 策略学不动。

诊断: 亲手加惩罚看它崩, 亲手删惩罚看它活

我做了一组对照实验(单变量): 把出界惩罚彻底删掉, 回到原始 reward。结果:

版本	训练前 eval	现象
出界惩罚 -1	-780	躺平, std 锁死 1.0
出界惩罚 -0.3	-275	ev 涨但 eval 卡死, std 不降
原始 reward(删惩罚)	+56	有正反馈, std 开始下降, 策略真的在学

根因: 出界惩罚制造了一个"不动 = 安全"的局部最优。而原始 reward 里, 草地本身也只 -0.1/帧(和赛道上"已访问 tile"一样), 所以"朝赛道方向挪"始终有正向梯度——**策略有方向可学, std 才能降。**

教训 2: reward 是最敏感的东西, 能不动就不动。 agent 会忠实地最大化你写的 reward——你写"出界扣分", 它就学"永远别动"。misspecified reward 的代价是灾难性的、反直觉的。这个坑后来反复警示我: 所有稳定化优先用不碰 reward的手段(clip、mb、lr、std 初始化), 而不是加惩罚项。

3. KL 爆炸与 critic 脱节: 冲高后崩溃的完整机制

回到原始 reward 后, 策略开始真正地学。eval 一路震荡向上, 多次冲到 400-700。但**每次冲高之后, 几乎都会崩**。我花了很久才拼出崩溃的完整机制, 它其实有两个不同的版本。

版本 A: KL 爆炸(epochs 复用 + 高方差)

早期 run(epochs=4, n_ep=2): 冲到 447 后, it107 kl=0.105、it109 kl=0.149、it119 kl=0.21——**连续 KL 爆炸**。策略被一路推离, critic 追不上, eval 447 $\rightarrow$ 233 $\rightarrow$ -79 崩死。
根因: epochs=4 复用同一批数据 4 遍 + n_ep=2 的高方差 advantage $\rightarrow$ 更新在同方向累积推动 $\rightarrow$ KL 滚雪球。

修复: epochs 降到 1(不复用), mb 加到 1024, n_steps 从 2000 提到 20000(n_ep 从 2 提到 20, 方差骤降)。这一组把 KL 爆炸压住了——但埋下了后面"mb 太大导致 std 不降"的隐患。

版本 B: critic 脱节崩溃(冲高 $\rightarrow$ return 尺度爆 $\rightarrow$ ev 变负)

后期 run: eval 冲到 676/702 后, 出现一种新崩法。
• vL(value loss) 从 5 飙到 **99**;

- ev 从 0.9 掉到 **负数(-0.06 甚至 -1.5)**;
- eval 崩到 -93,彻底卡死。

根因——critic "独自追大 return":

1. 冲高(如 702)→ return 尺度突然变大;
2. 此时若 KL 保护把 **policy 全冻住**(kl=0,不更新)——但 KL 保护只管 policy,不管 critic;
3. critic 仍在独自更新,想去匹配变大的 return,但拿到的是"旧策略采的数据"→ **估值目标与数据脱节** → critic 越追越乱 → ev 变负 → advantage 全错 → policy 被错误信号带崩。

教训 3:KL 保护太严会反噬。把 policy 全冻(kl=0)不是安全,而是让 critic 和 policy 脱节。信任域机制必须"松紧适度"——太松防不住爆炸,太紧制造脱节崩溃。这个平衡点非常难找(后面 $4\times/8\times/20\times$ 阈值都试过,各有各的崩法)。

4. log_std 的起点:std=1 是一切摇晃的根源

即使 KL 压住了,还有一个顽固的问题:**确定性策略(eval)总是"摇晃"**——过了第一个弯后,方向盘左右来回摆,越摆越大,最后摆进草地。

诊断链:摇晃 → 回正 → std 太大

逐帧看 gif,发现摇晃的本质是"**方向盘打了不回正**":进弯打方向 → 出弯该回正走直,但回正没做好 → 车持续偏 → 反向纠正 → 又过头 → 震荡发散。

再往下挖:为什么回正做不好?查文献发现这是有名字的问题——**steering oscillation / action jitter**,而根源之一是探索噪声太大。我的 `log_std` 初始化是 `zeros`,即 $\text{std} = e^0 = 1.0$ 。

std=1 的致命之处:方向盘输出 = 均值 $\pm$ 大范围随机(± 1)。即使均值学对了(该回正 → 输出 0),std=1 的大噪声也会把动作拽到 ± 1 ,回正被噪声毁掉。均值再准也没用。

修复:log_std_init = -1(std ≈ 0.37)

参考 SB3 调优过的 CarRacing 配置(`log_std_init=-2`),我把初始 std 从 1.0 降到 ≈ 0.37 :

```
self.log_std = nn.Parameter(torch.ones(act_dim) * -1.0) # std $\approx$ 0.37
# 配合下界 clamp(min=-2),防 std 坍缩到 0
```

立竿见影:换成 std=0.37 后,同样的训练:

- eval 方差从 ± 138 骤降到 $\pm 2-4$ (各赛道表现一致了);
- 动作精确,方向盘不再大幅乱抖;
- 20 万步 eval 就冲到 452,而且不崩(之前要 100 iter 且到点就崩)。

教训 4:初始化决定优化能不能走到好解。std 的起点(1.0)两头不讨好:降太快 → 过早收敛成脆弱的"一招鲜"(run02);不降 → 均值学不精、摇晃。而换个起点(0.37)直接绕过了"从大降到小"的危险过程——一开始就在小噪声下学精确控制。log_std_init 是个被严重低估的超参。

更深一层:后来意识到,std=1 的大噪声还会频繁把方向盘拽到"打死"这个危险极端 → 触发原地打圈 → 掉进"越转越偏"的吸收态。**在有"危险极端动作"的任务里,大 std 会主动把 agent 推进吸收态。**小 std 让动作稳定贴近均值,避开危险极端。

5. S 弯攻坚:从蹭草地到出弯回正的临门一脚

std=0.37 之后,策略开始扎实地攻克一个个弯。这一段是整个项目里最像"看着 agent 成长"的部分,值得记录它攻克 S 弯的完整轨迹(S 弯是这条赛道最难的点——连续反向弯,要"左-右-左"连续反打 + 全程控速)。

阶段	行为(拆帧)	解读
早期	一头栽进草地,卡死不出	完全不会 S 弯
中期	冲出草坪的位置逐次后移"半个小弯"	在逐段学 S 弯(软分段函数逐段拟合)
突破	高速蹭着 S 弯边缘穿行(road/grass 高频切换)	能过但靠"蹭",不干净
攻克	连续过 S 弯 + 第三弯,eval 从 452 跳到 676	掌握了"连续反打"的核心技能

关键洞察:入弯会了,出弯回正没学会

在"能过但不稳"的阶段,拆帧发现一个极精细的现象——问题**不在入弯,在出弯回正**:

- 某次 gif:出 S 弯后**回轮过大** → 反方向旋转;
- 另一次:出 S 弯后**回轮不足** → 同方向持续转圈。

两次都是"出弯方向盘回正没控准"——过头 / 不足。而这正好和数字对上:eval 方差大(± 237)、kl 反复爆,就是 policy 在"回正角度"上反复大幅调整。

教训 5:能力是一个技能一个技能长出来的,而且可以拆解到极细的粒度(入弯 vs 出弯回正)。看懂 agent 卡在"哪个技能的哪一段",比看 eval 分数深刻得多。这种"从 gif 读出 policy 在学什么"的能力,是整个项目最值钱的收获。

关于 ep_ret vs eval 的反转:攻克期还出现过 ep_ret(探索版)远高于 eval(确定性)的现象。关键:**eval 测 5 条固定赛道,ep_ret 测 20 条随机赛道**——两者跑的赛道根本不同,不能直接比"探索 vs 确定性"。这个混淆坑了我一次:曾用它错误论证"探索潜力",后来才反应过来是赛道难度差异。

6. checkpoint 的四个坑:工程细节如何白跑一整晚

攻克 S 弯、拿到 eval 676/702 后,我想从这个好策略"续训优化",于是开始折腾 checkpoint 恢复。这一段几乎全是工程坑——每一个都能让训练白跑几小时。它们不是算法问题,但同样致命。

坑 1:resume=False,续训变从头训

代码里 `train()` 默认 `resume=False`,而调用也是 `train()`。cell 把 best.pt 拷进去了,但 train 根本没加载——**从零初始化新网络训**。拷贝的 676 白拷。

修复: `train(resume=True)`。这个坑是 review 代码时抓到的——不 review 就白跑一晚。

坑 2:best.pt 没存 optimizer,KeyError

best.pt 只存了 `{model, it, eval}`,没存 opt。resume 代码硬读 `ck["opt"]` → **KeyError 直接崩**。

修复:容错加载(`if "opt" in ck`);并让此后的 best.pt 也存完整状态 `{model, opt, it, best_eval, hist}`。

余波:即便容错了,676 的 opt 永久丢失 → 从 best.pt 续训时 Adam 动量从零 → 前几步 warmup 步长偏大 → 差点把好策略推歪。**checkpoint 必须存完整状态(含 opt),否则无法真正无缝续训。**

坑 3:评测跨步数,best.pt 记录乱套

做课程学习(episode 步数从 2000 逐步收紧)时,一度让评测也用课程步数。后果:**2000 步下 eval 天然比 1000 步高**(能跑更多 tile)→ 一个"2000步才 900"的虚高分覆盖了"1000步 702"→ best.pt 记录的分数跨步数不可比,完全乱套。

修复:训练用课程步数,评测始终用 1000 步标准。这样 eval 可比、best.pt 一致。

坑 4:minibatch KL 保护是"假的"

为防 KL 爆炸,我加了"minibatch 级 KL 保护"。但 review 时发现它**根本没起作用**:

1. KL 检查放在 `opt.step()` 之后——那一步的更新已经生效,拦不住;
2. 检查用的是 step 之前算的旧 ratio,测不到"这次更新造成的 KL"。

正确实现(step 后算真实 KL + 超阈值回滚该 minibatch):

```
old_params = [p.detach().clone() for p in model.parameters()] # step前存参数
opt.step()
with torch.no_grad(): # step后重新前向
    new_logp2, _, _ = model.ev(obs[b], acts[b])
    logr2 = new_logp2 - old_logp[b]
    kl_real = (logr2.exp() - 1 - logr2).mean().item() # 真实KL(k3估计)
```



```

if kl_real > THRESH * target_kl:
    for p, old in zip(model.parameters(), old_params):
        p.copy_(old)
    # 回滚这次step
    early_stop = True; break

```

教训 6:"看着对但逻辑错"的代码,是 RL 里最坑的。训练一次几小时,一个 `resume=False` 或"假的 KL 保护"能让你白跑一整晚还不知道为什么。养成习惯:改完先 review 关键逻辑(加载 / resume / 路径 / 保护机制),再跑一两步确认加载对了(开头 eval 是不是 676),才挂长训练。

阈值调参的三次崩法(KL 保护松紧之难)

即便 KL 保护实现对了,阈值(几倍 target_kl)也极难调:

阈值	结果
4× (0.12)	从 checkpoint 续时,Adam 首步 bias-correction 大步 → 每个 minibatch 都超阈值被回滚 → 全冻,策略卡死不更新 (ev/kl/vL 全 0)
20× (0.6)	太松,kl=0.08 的"小更新但方向坏"过得去 → 一步把 702 推歪成"沿草边走"
8× (0.24)	配大 batch(2048)时,单次更新推动更狠 → eval 方差爆到 ±431,it63 崩死 -193

关键认识:KL 保护防的是"飘太远(大 KL)",但防不了"小步(kl=0.08)滑向坏策略"——因为 kl 不大,不触发。把 702 推歪成"沿草边走"的那次更新,kl 才 0.08。这说明单靠 KL 信任域治不了 reward hacking 式的缓慢滑移。

6.5 更多致命坑:没单独成章,但一样白跑一晚

除了前面的大坑,debug 后半程还踩了一串"续训/工程"相关的失败。它们单看不起眼,但每一个都让训练白跑,而且环环相扣。

坑 5:warmup 卡死——从 best.pt 续 + 新 opt + 严阈值 = 完全冻结

从 676 的 best.pt 续训,前几步(it33-35)出现诡异的"全冻":**ev / kl / clip / vL 全是 0,eval 死守 676.7 一动不动。**

根因:best.pt 没存 opt → 新 Adam 动量从零 → 首步 **bias-correction** 让有效步长偏大 → 每个 minibatch 都超 4× 阈值 → 被 KL 保护全部回滚 → 一整轮 update 没有一次成功 → 策略纹丝不动。

诡异之处:降 lr(1e-4→3e-5)都救不了——因为 Adam bias-correction 与 lr 无关,首步照样大。

坑 6:续训第一步就把好策略推歪

换个阈值($20\times$)后不卡死了,但**续训第一步(it44)kl 才 0.08,就把 702 推歪成"沿草边走",eval 直接 53。**

根因:702 是个"尖峰"——周围策略空间陡峭,一个不大的更新($kl=0.08$)就滑离。而且这次更新方向朝着"贴草边"这个 hacking 吸引子。**KL 保护防不了这种"小步滑向坏策略"。**

坑 7:一次改 4 个变量,归因不出

有一版(v9)一次性改了 **4 件事**:训练早停 + clip $0.2\rightarrow 0.1$ + minibatch KL 保护 + 从 best.pt 续(新 opt)。结果崩了,但**完全归因不出是哪个改动的锅**——因为一次动了四个变量,其中"早停"方向还是反的。

教训:这是自己违反单变量原则的现行反面教材。回退到"以 v8 为基线,只改一件(KL 保护)"才理清。

坑 8:eval_every=1 改变了训练轨迹,复现不出稳策略

为了让"课程学习状态机"更灵敏,把评测从每 10 轮改成**每轮(eval_every=1)**。副作用:**训练轨迹变了**——之前 eval_every=10 能跑出 it36 的"方差小(± 47)稳策略",改成每轮后方差一直大($\pm 150-280$),**始终没复现出那个稳策略**,课程触发条件(方差 <50)永不满足,然后一路震荡崩死到 -93 卡死。

根因:评测本身不改训练,但改 eval_every 连带影响了别的(采样节奏 / 随机数流),轨迹就偏了。RL 对这种"看似无关"的改动异常敏感。

坑 9:课程触发条件太严,稳策略被挡在门外

课程触发设了"eval >550 且 方差 <50 "。结果 it62 的 eval 480、方差 **53**——**就差一点点没触发**($480<550$ 且 $53>50$),课程没启动,下一步就崩了。稳策略被过严的门槛挡在外面。

坑 10:"扰动 + 弃用 opt"实验的意外发现

想用"加载 702 权重 + 弃用它的 opt(全新 opt 当扰动)"来跳出"续训就崩"的困境。结果 gif 还是"贴草坪"——**但这次的发现很关键:it44 几乎没更新($kl\approx 0$),gif 却已经贴草坪,说明贴草坪是 702 权重本身在某些赛道(seed[0])的固有行为,不是训练崩出来的幺蛾子。**

意义:eval 702(5 条平均)靠其他赛道撑着,seed[0] 这条它本来就贴草坪。**微调(opt/lr/kl)治不了策略的固有弱点,得实质重训。**

遗留未修:truncated 混进 dones 的 bootstrap 偏差

一个从最早版本就存在、全程未修的 bug:CarRacing 的 TimeLimit 是 1000 步,而每个 rollout 必然撞上 1-2 次 truncation。**把 truncated 当成 terminated 处理,会在 GAE 的 bootstrap 上引入偏**

差(超时截断时其实还有未来价值,不该按"回合结束、无未来"来 bootstrap)。这个偏差一直在背景里,是已知但尚未处理的技术债。

这一串坑的共同教训:RL 的后期 debug 很大比例不是"算法"而是"续训 / checkpoint / 随机数流 / 触发条件"这类工程细节。它们**看着无关紧要,却能让训练悄无声息地跑偏或白跑**。而且它们环环相扣:opt 没存 → warmup 大步 → 触发严阈值 → 全冻;或放松阈值 → 又滑向 hacking。RL 的稳定性是被这些细节层层决定的,没有一个可以掉以轻心。

7. 2000 步的么蛾子:步数宽松如何喂养 reward hacking

攻克 S 弯拿到 702 后,一个新目标浮现:eval 676/702 常常是"半途冲草地之前攒的分",并非真的跑完整圈。为了让它跑完整圈,我把 episode 步数从 1000 加到 **2000**,想给它更多时间。

预判(后来应验):当时就直觉"2000 步会有么蛾子"。理由:步数宽松 → 提速压力小 → agent 可以"慢慢磨也能攒分",没有动力跑快。

么蛾子逐帧记录

从走线完美的 iter60(1000 步下 99% 全程中线)续训到 2000 步,行为演化:

iter	占比	速度	行为
61	52%	—	跑一半冲草地卡到底(318 帧全草地)
62	38%	—	更早冲草地,eval 714 是冲前攒的分
63	75%	2.4	学会脱困: 进草地 106 帧后能开出来
64	99%	1.8	走线完美: 全程中线,只开头进草地
65	99%	1.8	稳定在"慢而龟",2000 步跑不完一圈
66	—	—	滑向"沿草边贴走", eval 517 → 111

么蛾子的两个形态:

- 慢而龟**(it64/65):为了不冲草地,慢到 1.8——2000 步都跑不完一圈,eval 只有 490–517。步数宽松让它"偷懒慢跑"。
- 沿草边贴走**(it66):从"内切边缘还在赛道"滑成"压草地贴边蹭",eval 崩到 111。这是 reward hacking——沿边界慢慢蹭 tile,是个"够用但低效"的吸引子。

更换配置(n_steps 40000 + mb 2048,想用大数据大 batch 稳住)也没救,反而崩得更快:大 batch 单次推动更狠,it62 方差爆 ±431,it63 崩死 -193。

教训 7:episode 步数是个被忽视的、强力的隐性超参。2000 步(宽松)是么蛾子的温床——return 尺度变大(易崩)、提速压力消失(偷懒)、给足了"沿边慢蹭"的空间(reward hacking)。反而 1000 步逼得紧,提速压力大。宽松的约束会喂养懒惰和取巧,这是 RL 里一个反直觉但深刻的规律。

7.5 训练中的"吸引子动物园"

整个 debug 过程中,agent 掉进过形形色色的**吸引子**——一旦落入,就稳定地重复某个怪异行为,而且极难自己出来。这些吸引子是 RL 最生动的失败形态,值得单独归档。它们本质上是**局部最优**(参数空间)与**状态吸收态**(状态空间)的叠加,可以分成三类。

第一类:探索失效型(掉进去,随机也逃不出)

① 草地原地打转(冲出赛道后)

车冲出赛道进草地,原地画圈,身后一圈圈螺旋轨迹。**越转越偏离赛道,离能得分的地方越来越远**,于是越来越难回去——这是个"越陷越深"的吸收态。

为什么随机探索逃不出:回赛道需要"朝特定方向连续开一段"(小概率精确事件),草地一大片,随机动作多半还在草地里 → 撞不中"回赛道"。std 大小都救不了(std 大也是乱转,撞不中)。

② 起点/原地打转(it35 一次 kl=0.68 猛更新后)

从起点就原地转圈,方向盘打死不回正。这是策略被一次超猛更新(kl=0.68)推到了"打死方向"的极端 → 车持续同向转 → 原地打圈。

第二类:reward hacking 型(够用但取巧,有正反馈喂着)

③ 沿草地和路的边缘走(贴草边)(it66)

车贴着 road/grass 交界线跑,一半蹭赛道边缘、一半压草地,慢慢蹭 tile。占比看着不低,但净分低(时间惩罚 > 蹭到的 tile 分)。

为什么顽固:贴边能蹭到一点正反馈(碰到边缘 tile),KL 保护又拦不住这种"小步(kl=0.08)缓慢滑向贴边"的更新——不是过度更新,是缓慢滑移。

④ 横跳蹭 tile(it35 崩后,road/grass 切换 18 次)

与"贴草边"不同:这是**高频反复进出**赛道边缘——蹭一段赛道(RRRR)→ 出去一下(...)→ 再蹭 → 规律循环。均匀的横跳模式,稳定地往前蹭分,但一半时间在草地,低效。

第三类:过度保守型(偷懒/躺平,被约束逼出来的)

⑤ 躺平不动(出界惩罚 -1 那次)

学会了原地不动——出界 -1 太重,"不动 = 不出界 = 最安全"。std 从 1.0 纹丝不动,策略锁死在这个吸收态。eval -1045。

⑥ 停在赛道上不动 / 慢而龟(2000 步,it64/65)

走线完美(99% 全程中线)但慢到 1.8——2000 步都跑不完一圈。步数宽松让它"慢慢磨也能攒分",没有提速动力。这是被"宽松步数"喂养出的偷懒吸引子。

第四类:控制失稳型(回正没学好的震荡)

⑦ 打死方向画圆(it30)

出 S 弯后方向盘固定在某个非零角度不回正 → 车稳定画一个圆。回轮"没回够"。

⑧ 反向旋转(it29)

出 S 弯后回轮"过大"→ 打过头 → 反方向转起来。⑦⑧ 是同一问题(出弯回正没控准)的两个方向。

吸引子的统一教训:

- **探索失效型**(打转)—— 掉进吸收态后,靠加大探索(std/均匀分布/换 Beta)都极难逃出,因为"回赛道"是小概率精确事件。
- **reward hacking 型**(贴草边/横跳)—— 有微弱正反馈喂着,KL 保护拦不住缓慢滑移;根治要靠训练 cost(贴边扣分)+ 评测用真实 reward。
- **过度保守型**(躺平/慢而龟)—— 由过重的惩罚或过松的步数约束喂养;修复要动约束本身。
- **控制失稳型**(画圆/反转)—— 精确控制(回正)没学会;小 std + 更多训练。

共同的解法哲学:预防 > 逃逸。掉进吸引子后逃出来极难;不如用精确控制(小 std、log_std_init)从一开始就别掉进去。这是 CarRacing 反复教我的一课。

8. 两个迟到的洞察:采样与 reward 的重新理解

debug 到这里,踩坑已经从"算法参数"上升到"训练机制设计"。两个洞察浮现出来,它们其实各自对应 RL 里一个正经的技术方向。

洞察一:智能采样——别让吸引子样本主导训练

一次 rollout 20000 步、约 20 局,如果大部分样本都陷在同一个吸引子(草地打转、沿草边、原地不动),训练就会被这些"死循环样本"主导 → 学歪。设想的改进:

- 对能自己跳出吸引子的样本(脱困、突破)给更高 **sample weight**;
- 允许 dirty sample(噪声/失败样本,有"别这样"的信息),但噪声不能成为主流;

- 让各种噪声样本进入主采样 → 增加训练鲁棒性,甚至不必死限 std 在 0.35 附近。

对应的正经方法:这本质是 **Prioritized Experience Replay(优先经验回放)** + **样本多样性/去偏** 的思想——只是优先标准换成了"是否跳出吸引子"。核心是:采样层面的 exploration-exploitation 平衡,可能比死限制 std 更本质。

洞察二:训练 cost ≠ 评测 reward(之前我们混了)

整篇 debug 里,我一直抗拒"改 reward",因为一改,评测分也变,分数就不可比了。但根本症结是——**训练用的奖励和评测用的指标,本就应该分开:**

- **训练:**可以用 **shaped reward / cost function**(如"车到草地边缘 → reward 降低")来引导学习(别贴边);
- **评测:**始终用**系统的真实得分**(原始 reward),衡量真实水平。

这解决了长期的顾虑:之前不敢改 reward,是因为"训练和评测用同一个 reward,一改就乱"。正确做法是——**草地惩罚只用于训练(cost 引导别贴边)**,**评测永远用原始 reward(真实分、可比)**。这样既能用 cost 治 reward hacking(贴边/沿草边),又保持评测客观。这正是 **reward shaping 的正确用法**:训练可以"作弊"引导,评测必须诚实。之前把训练 cost 和评测 reward 混为一谈,是一个概念性错误。

9. 方法论沉淀

34 步 debug 长征沉淀下来的,不是某个超参,而是一套可迁移的方法论:

- **数字 + gif 双读,冲突时信 gif。** eval 452 是打转攒分、714 是冲前攒分、99% 占比的"慢而龟"——这些真相都藏在行为里,数字看不出。进一步:拆帧量化(赛道占比、road/grass 切换次数、冲出草坪的位置、速度分段)比肉眼更准。
- **单变量原则。**一次只改一个,才能归因。有一版一次改了 4 件事(早停 + clip + KL 保护 + 新 opt),结果脏得完全归因不出——这是自己违反原则的教训。
- **每步存档。**模型不大(19MB),5TB 空间随便存。正是"每轮存 iter checkpoint"救回了 iter60/iter64 这些"best.pt 没记录但走线完美"的好状态——它们后来成了可复用的上游初始模型。
- **checkpoint 存完整状态。**model + opt + it + best_eval + hist,一个都不能少,否则无法真正无缝续训。
- **先 review 再跑。**训练一次几小时,resume=False / 假 KL 保护 / 评测跨步数这类坑,review 五分钟能省一整晚。
- **预防 > 逃逸。**掉进吸收态(草地打转)后靠探索逃出来极难;不如用精确控制(小 std)预防,从一开始就别掉进去。

这个小里程碑的坐标:从单环境、高斯策略、手写 PPO 出发,拿到过 eval 702、训出 iter60/64 "99% 全程中线"的走线完美策略——在 HuggingFace CarRacing 排行榜属中上游(大量 PPO 卡在 400-

500,800+ 需 LSTM/Beta + 8 并行 + 400 万步)。更重要的是:踩遍了 reward misspec、KL 爆炸、critic 脱节、std 起点、reward hacking、checkpoint 四坑,并从中长出了"看懂 RL 学习过程"的能力。

10. 最终成果:从崩溃到 iter60 "走线完美"的完整路径

踩遍所有坑之后,最好的策略并非某次冲高的虚高分,而是 **iter60**——一个 99% 全程走在赛道中线、泛化能力强的稳定策略。它是怎么在一片崩溃与幺蛾子中被"打捞"出来的?这里复盘完整路径。

关键转折链

步骤	动作	效果
1	删掉出界惩罚,回原始 reward	策略有方向可学,std 开始下降
2	log_std_init = -1 (std $\approx$ 0.37)	动作精确,eval 方差 $\pm 138 \rightarrow \pm 2-4$,不再摇晃
3	epochs=1 / mb=1024 / n_steps=20000	压住 KL 爆炸,ep_ret 方差骤降
4	攻克 S 弯(蹭草地 $\rightarrow$ 回正)	eval 冲到 676$\rightarrow$702
5	从走线好的中间态 续训 (而非从崩溃点)	学会草地脱困(占比 75%)
6	继续训到 iter60/64	占比 99%,全程中线,只开头进草地

"打捞"iter60 的关键工程决策

每步存档救了它。 iter60 的 eval(1000 步、5 条固定赛道)只有 285——因为它**慢而稳**(走线完美但速度低,1000 步跑不完整圈),所以 best.pt(按 eval 最高存)根本没记录它。是"每轮单独存 iter{it}.pt"这个决定,把这个"分不高但走线完美"的宝贵策略留了下来。

后来用拆帧量化验证:iter60 **占比 99%、road/grass 切换仅 1 次、全程贴内切线(最短路径)**——是理想的"上游初始模型"。而 2000 步下评测,它的真实水平其实高达 **eval 630**(慢策略给足步数就能跑完更多)。

为什么 iter60 比 eval 702 那个"更好":

- 702 方差 ± 200 ,某些赛道(seed[0])其实**贴草坪**——高分靠其他赛道撑;
- iter60 方差极小,**各赛道一致地全程中线**——走线是"对"的,只是慢;
- **走线正确(难) + 慢(易补) > 快(有了) + 走线错(难改)**。iter60 就差"提速"这一个可补的短板,是更好的下游起点。

成果

- **最好策略**:iter60,占比 99% 全程中线,2000 步 eval 630,ep_ret 448(20 局随机赛道,泛化好)。
- **定位**:单环境 + 高斯策略 + 手写 PPO,在 HuggingFace CarRacing 排行榜属**中上游**(大量 PPO 卡在

400-500;800+ 需 LSTM/Beta + 8 并行 + 400 万步)。

- **下一步(下游探索,均从 iter60 暖启动):**课程学习提速、多模型加权集成(速度互补)、CNN feature 复用(如 + GBDT 输出动作头)、换架构(LSTM/Beta)、以及本篇末尾提出的智能采样与训练cost/评测reward分离。

产物

训练全程产物落在 Google Drive: [MyDrive/carracing/<RUN>/](#) (gifs / ckpt / metrics.csv),支持断点续训、每步存档。

- **最好策略的评测 gif**(99% 全程中线):
drive.google.com/drive/folders/1Vy7AwiFSUMyI7_GwcNyklGKuMeueN6dS
- **最好模型 checkpoint**(iter60):
drive.google.com/file/d/1dwzW1EzzCK5QOY11JADtMjQ8pAILBc59

收尾的一句话:这一路最有价值的,不是拿到 eval 702,而是训出 iter60 这个"走线完美"的策略——并且看懂了它是怎么从一片崩溃、么蛾子、吸引子里,靠"原始 reward + 小 std + 每步存档 + 拆帧读行为"一步步被打捞出来的。分数会过时,但这套"在混乱中定位好策略"的能力,是可迁移的真本事。

— Part 4 完。下一站:智能采样 与 训练cost/评测reward 分离的落地实验。 —